

ARQUITETURA DO SISTEMA

EdTech Academy — PlataformaEnsino (Sistema Acadêmico Integrado)

Projeto Integrado Multidisciplinar — 4º Semestre · Análise e Desenvolvimento de Sistemas · Universidade Paulista — UNIP

Krigor de Sousa Nasare · Heitor Henrique Pereira Nadir · Maria Isabel Branco Iterce da Silva · Nicolas Pimentel dos Santos

1 VISÃO GERAL

A PlataformaEnsino (marca comercial **EdTech Academy**) é um sistema acadêmico integrado que atende quatro perfis de usuário — Aluno, Professor, Coordenador e Admin — cobrindo o ciclo completo de uma matrícula: catálogo público de cursos, solicitação e aprovação de matrícula, publicação de conteúdo didático, aplicação e correção de avaliações, acompanhamento de progresso e emissão de certificado com verificação pública.

O projeto nasceu como uma aplicação web única e evoluiu, ao longo do curso, para uma arquitetura multiplataforma: o mesmo backend passou a atender também um aplicativo mobile e, futuramente, uma versão desktop, sem duplicar regra de negócio. Essa evolução — e as decisões técnicas por trás dela — é o assunto deste documento.

2 ARQUITETURA EM CAMADAS DO BACKEND

O backend segue uma separação clássica em camadas, implementada em C# com ASP.NET Core e Entity Framework Core. Cada camada tem uma única responsabilidade e só conhece a camada imediatamente abaixo dela, o que mantém a regra de negócio isolada da forma como os dados chegam (HTTP) ou são persistidos (SQL Server).

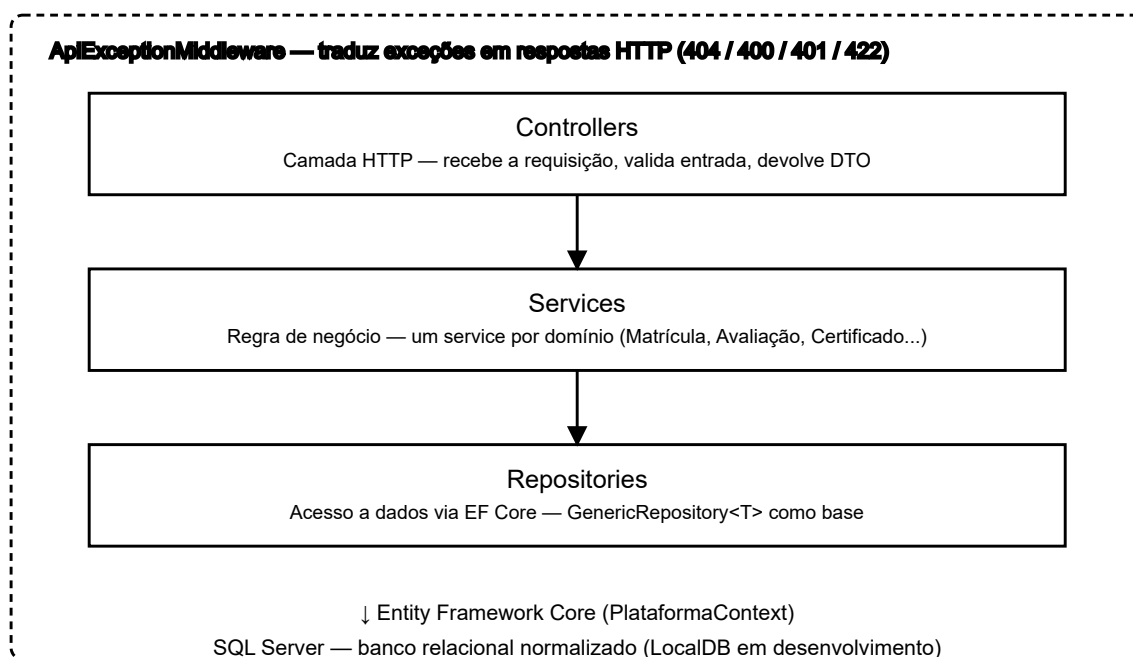


Figura 1 — Fluxo de uma requisição pelas camadas do backend, do HTTP até o banco de dados.

Duas camadas adicionais dão suporte às quatro principais: `DTOs/`, que define os contratos de entrada e saída da API — nenhum endpoint devolve uma entidade do Entity Framework diretamente, evitando expor campos internos ou navegações inteiras sem controle — e `Common/`, que reúne lógica transversal reutilizada por várias camadas, como geração de código de matrícula único, extração do usuário autenticado a partir do token e mapeamento dos campos comuns de `Usuario`.

3 MODELO MULTIPLATAFORMA

A decisão de arquitetura mais relevante do projeto foi como estender uma aplicação web já pronta para mobile e, futuramente, desktop, sem reescrever a regra de negócio nem duplicá-la em cada cliente. A alternativa descartada foi reescrever o backend em outra stack pensada para múltiplos clientes — o que jogaria fora a entrega acadêmica em C#/.NET já validada. A solução adotada foi introduzir um **gateway**: um serviço Node.js/Express que fica na frente da API .NET e funciona como ponto único de entrada para qualquer cliente.

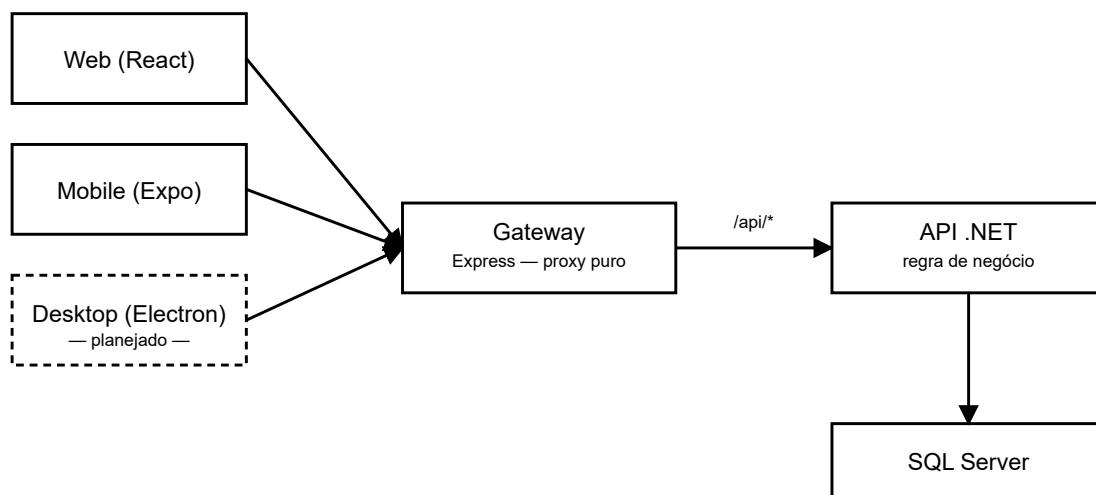


Figura 2 — Web, mobile e o futuro desktop conversam com o mesmo gateway, que repassa tudo para a única API que acessa o banco.

O gateway não tem lógica de negócio própria: ele apenas repassa as rotas `/api/*` (e `/uploads/*`, usada pelos arquivos enviados) para a API .NET, incluindo o cabeçalho `Authorization`. Essa simplicidade é intencional — se o gateway algum dia precisar agregar ou simplificar respostas para o mobile, é ali que essa lógica entraria, mas hoje ela não existe porque não é necessária. O desktop, quando implementado, vai embalar com Electron o mesmo código do frontend web, e por isso herda a integração com o gateway automaticamente, sem nenhum código novo.

O aplicativo mobile (Expo / React Native) foi construído com as mesmas convenções do frontend web — tanto que, ao identificar que a lógica de autenticação estava sendo mantida em duas cópias divergentes (uma por cliente), o núcleo de comunicação com a API e de gerenciamento de sessão foi extraído para um módulo único, `shared/`, reaproveitado pelos dois projetos. A única diferença entre eles é o adaptador de armazenamento local — `localStorage` no navegador, `AsyncStorage` no React Native — parametrizado como argumento das funções compartilhadas.

4 AUTENTICAÇÃO E SESSÃO

A autenticação usa **JWT** (JSON Web Token) assinado com HS256. No login, o `AuthController` delega a validação de credenciais ao `AuthService`, que devolve um token de acesso — válido por 120 minutos, contendo claims de identidade e papel do usuário — e um **refresh token** opaco, persistido no banco (tabela `RefreshTokens`).

Quando o token de acesso expira, os clientes (web e mobile, via o módulo `shared/`) detectam a resposta `401` e chamam automaticamente `POST`

`/api/auth/refresh` antes de repetir a requisição original — o usuário não percebe a renovação. Cada uso do refresh token o invalida e emite um par novo (**rotação**): um token de renovação vazado ou reutilizado indevidamente só funciona uma única vez, e a tentativa seguinte é recusada. O logout explícito revoga o refresh token no servidor, não apenas localmente.

Nota: a rotação de refresh token e a renovação automática compartilhada entre clientes foram adicionadas depois de uma revisão de arquitetura que identificou que a sessão de 120 minutos, sem renovação, era tolerável numa aba de navegador mas prejudicial num aplicativo instalado (mobile), que fica aberto por muito mais tempo.

5 MODELO DE DADOS

O domínio é modelado em torno de uma hierarquia de usuários (`Usuario` abstrato, com `Aluno`, `Professor`, `Coordenador` e `Admin` como perfis concretos, mapeados por Table Per Type no Entity Framework Core) e de uma cadeia acadêmica principal:

`Curso → Módulo → Conteúdo Didático`

`Curso → Turma → Matrícula (Aluno + Curso + Turma)`

O progresso do aluno é rastreado em granularidade de conteúdo, módulo e curso (`ProgressoConteudoAluno`, `ProgressoModuloAluno`, `ProgressoCursoAluno`), e o sistema de avaliação segue um fluxo de publicação em duas etapas — um banco de questões reutilizável (`QuestaoBanco`) é selecionado e publicado numa avaliação específica (`QuestaoPublicada`), da qual o aluno faz tentativas (`TentativaAvaliacao → RespostaAluno`). A modelagem completa de tabelas, tipos e relacionamentos está detalhada no dicionário de dados do projeto (`dicionarioDados.html`), que acompanha este documento.

6 STACK TECNOLÓGICA

Tabela 1 – Tecnologias por camada do sistema

Camada	Tecnologia	Responsabilidade
API	<code>.NET 10 / ASP.NET Core / C#</code>	Regra de negócio, autenticação, única camada que acessa o banco
Persistência	<code>Entity Framework Core / SQL Server</code>	Mapeamento objeto-relacional e banco relacional normalizado
Gateway	<code>Node.js / Express</code>	Proxy reverso único para web, mobile e desktop
Web	<code>React / Vite</code>	SPA com roteador próprio, servida como arquivo estático pelo ASP.NET em produção
Mobile	<code>Expo / React Native</code>	Aplicativo nativo (iOS/Android) e web, compartilhando a mesma API
Desktop	<code>Electron (planejado)</code>	Empacota o mesmo frontend web numa janela nativa
Código compartilhado	<code>shared/ (JavaScript puro)</code>	Cliente HTTP e gerenciamento de sessão reaproveitados por web e mobile

Fonte: elaborado pelos autores a partir da configuração atual do repositório.

7 BOAS PRÁTICAS DE ENGENHARIA APLICADAS

Ao longo do desenvolvimento, o projeto passou por rodadas de revisão técnica voltadas a nomenclatura semântica, eliminação de código duplicado, organização de camadas, responsividade, acessibilidade e SEO. Como resultado, lógica que antes se repetia em vários pontos do backend — geração de código de registro único, validação de e-mail e CPF, extração do usuário autenticado a partir do token, mapeamento dos campos comuns de usuário — hoje vive em um único lugar (pasta `Common/`) e é reutilizada, reduzindo a chance de um mesmo bug precisar ser corrigido duas vezes de forma independente.

O mesmo princípio orientou a extração do módulo `shared/` entre web e mobile: a duplicação de código entre clientes é tratada como um risco técnico, não apenas uma questão de estilo, porque duas cópias da mesma lógica tendem a divergir silenciosamente com o tempo — o que de fato ocorreu antes dessa extração.